Boxu GDD

COMP710 Individual Game Project

16/04/26

Bardia Khalifeh
23196422

1. Game Overview

Boxu is a 2D top-down stealth strategy game, which places an emphasis on planning, observation, and precision. The player must infiltrate a guarded area, complete an objective, and reach extraction without being detected, and whilst being in a box the entire time affecting their vision. Unlike action stealth games, Boxu removes combat entirely, meaning every mistake matters and detection results in failure.

2. Game Inspiration

Boxu is inspired from the stealth focused challenge modes such as VR Missions from Hideo Kojima's Metal Gear Solid 2. Particularly their emphasis on route planning, restricted information, and precision based gameplay.

Rather than focusing on combat, Boxu prioritizes observation, movement control, and environmental awareness to create tension-driven stealth gameplay.

3. Core Design

Tension built by Vulnerability

The player cannot fight enemies directly. Every mistake carries consequences, which creates a natural tension through vulnerability rather than combat.

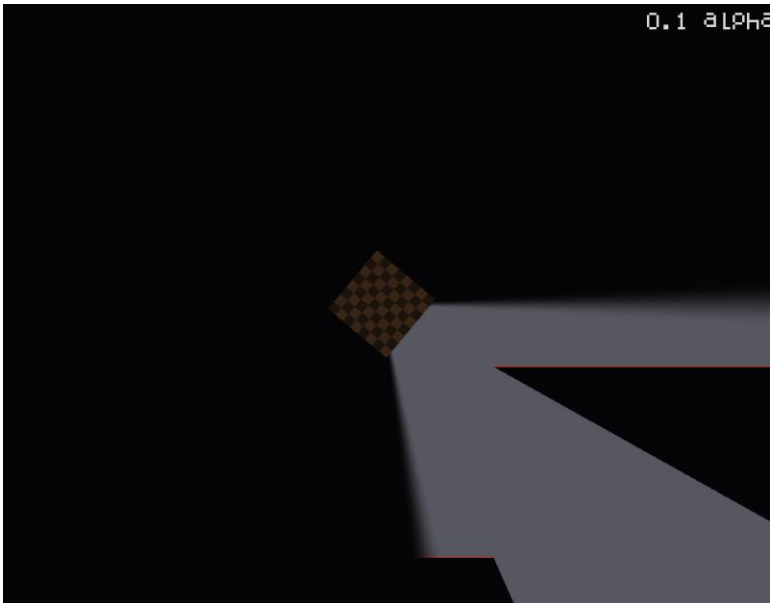
Sound and Risk Management

Movement speed directly affects noise generation. Player's must balance speed and stealth carefully during navigation.

Limited Awareness

The player has restricted vision, forcing the player to rely on observation, memory, and positioning rather than complete map awareness.

Figure A: Players restricted vision, wall affecting the player's vision.



Replayability Through Uncertainty

Enemy behavior, patrol flow, and player decision making create unpredictable scenarios that encourages repeated playthroughs.

Sound and Noise

Walking into the wall generates a noise; running into walls would generate noise covering a larger radius compared to walking into walls. Walking will generally have a lower radius of sound generated compared to running with SHIFT.

Enemy Behavior

Enemy patrols and may choose to stay in a position longer if they are investigating (player nearby). They wake on sound generated from the player. Once they are triggered to chase the player either by seeing them clearly or hearing them clearly (within a radius generated by the player's noise), then the enemy will chase of the player until they've either lost them or caught them.

4. Game Rules and Mechanics

Mission structure

Player starts in a dark, quiet section of the map. They are given a brief of their objective on the screen. How the player accomplishes said objective is up to them leading to replay ability. After the objective, the player must extract without getting caught, ideally without ever alerting the enemy of their presence. Player wins if objective is complete and reached extraction without being captured. Player loses if they are captured.

Player Movement

Player moves with WASD and uses the mouse to control the direction they will be facing. Sprinting will allow faster movement at the cost of more noise generated whilst moving faster. This includes when the player hits the wall; noise generated will be much louder (covering a wider radius) compared to walking into a wall. Sprinting must be used wisely.

Detection and Enemy State System

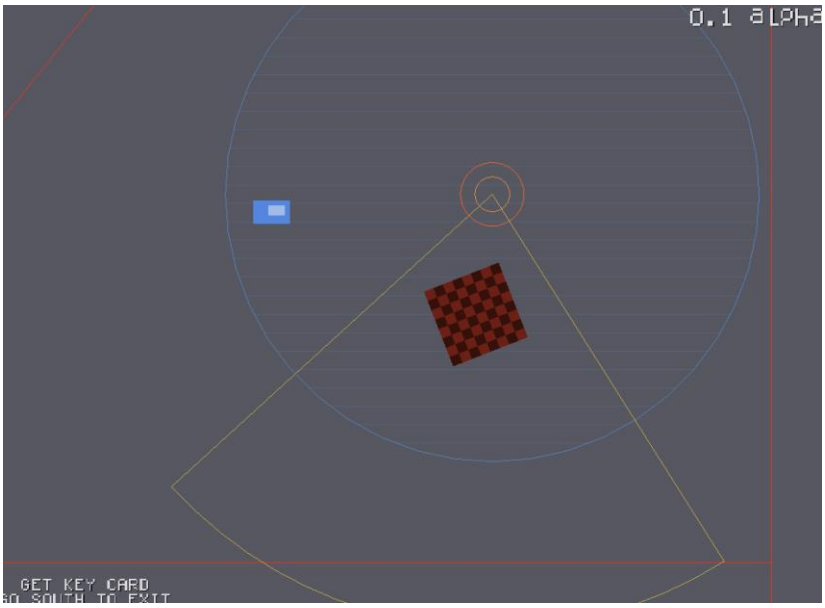
Enemy behavior is controlled using a state based system consisting of:

- Investigate
- Chase
- Search

Enemies detect the player through vision and sound.

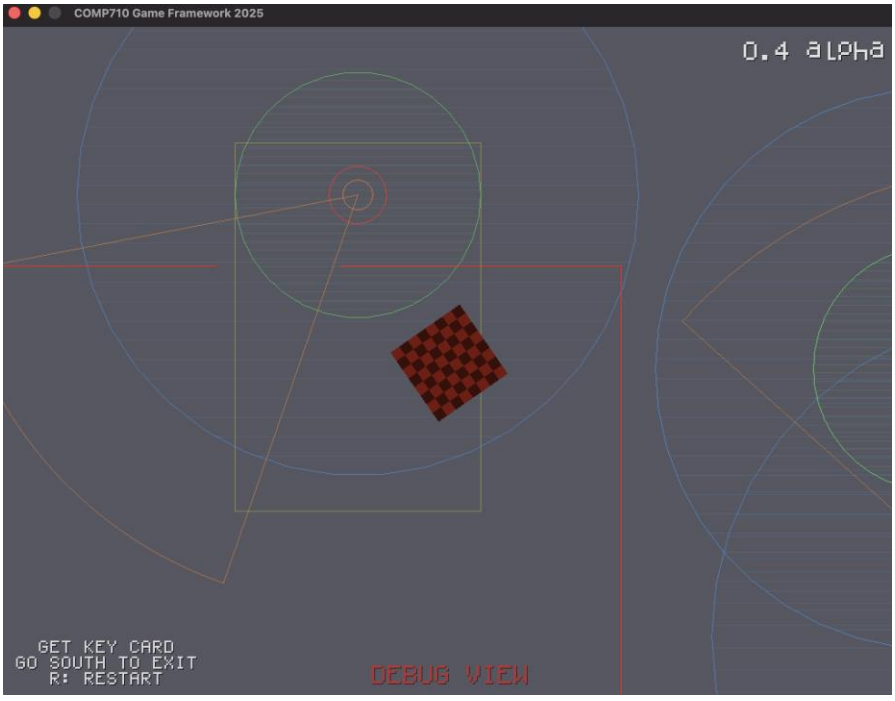
If the player enters an enemy's(red circle) line of sight (yellow cone outline) without any obstructions, the enemy immediately enters into a chase state. If the player generates noise nearby, enemies will also be triggered to investigate the source of the sound. If visual confirmation is obtained, the enemy transitions into chase.

Figure B: Early demo of enemy chasing player, player within enemy's sight and hearing radius.



Enemies do not immediately stop chasing once the player breaks their line of sight. Instead, a persistence timer allows enemies to continue their search briefly prior to returning back to a patrol behavior. This would make the experience more believable and tense, especially for a stealth gameplay.

Figure C: Further refined enemy's hearing radius. Green for quiet noises, Blue for loud noises.



5. Key Algorithms Governing Gameplay

Noise radius check

When the player collides with a wall, a noise is generated at the impact position. Enemies within the event radius may wake up or react.

Wall collision

Player generates a small radius of noise by walking into a wall, or a large radius of noise by running into a wall.

Detection & E.S.S (Enemy State System)

If the player is within the hearing radius of the enemy, yet a direct line of sight cannot be established, too much noise generated by the player would have the enemy investigate the last known location of the player.

On the contrary, if enough of the player's hitbox is within the vision-cone of the enemy, and there are no obstacles blocking the way, this would trigger an immediate chase against the player.

Player's vision cone

The player's field of vision is restricted to a forward-facing cone, which imitates the limited visibility of being inside a box.

The player must use the mouse and rotate it around the character to observe their surroundings. This mechanic reduces awareness and adds an additional layer of difficulty.

6. Level Design Philosophy

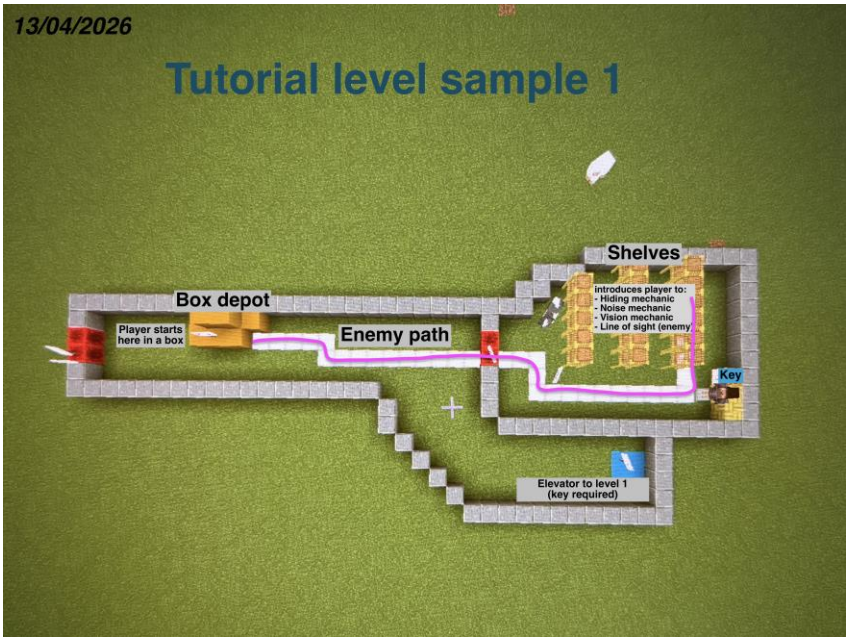
Levels are designed to encourage observation, route planning, and tension management.

Figure D: First sketch of Level 1

Narrow corridors increase the risk of collision noise, and close enemy encounters, while open spaces increase visibility risk and reduce available cover.

This helps create a balance between movement, efficiency and stealth safety.

Figure E: Prototype of Level 1 pre-sketch.



Early level layouts for Boxu were prototyped using Minecraft as a spatial visualization tool prior to being recreated within the game itself.

This allowed rapid experimentation with:

- Corridor spacing
- Line of sight management
- Obstacle placement
- Room scale
- Stealth route planning

Using a sandbox environment helped evaluate how confined spaces, corners, and environmental obstacles would affect stealth gameplay tension before implementing the layouts directly into the GP framework.

The final in-game layouts were then adapted and simplified to better suit the top-down perspective and gameplay requirements of Boxu.

7. Control Scheme

Input	Action
W, A, S, D	Move player (Up, Left, Down, Right)
Mouse	Control the facing direction
Left Shift	Sprint
H	Toggle debug mode
R	Restart level
ESC	Exit game

8. U.I (User Interface) design

The user interface is deliberately minimal to maintain the player's immersion and avoid distractions.

The interface includes

- Player objective displayed clearly
- Win and loss overlays

The Debug view

- Clear and red pulsing text written to inform the user that they are in debug view.
- Shows enemy hearing radius (a range for small noises e.g walking/walking into walls, and loud noises e.g running/running into walls. This also includes patrolling areas of enemies (yellow rectangle).
- Shows noise generated by player and color coded (e.g green for small noise, and blue for loud noise).

9. Asset List

Sprites

- Player (box character)
- Enemy guards
- Environment walls
- Access card
- Extraction zone

Audio

- Footstep sound effects
- Wall impact sound
- Interaction sound

Visual Effects

- Noise pulse circles
- Vision cone
- Debug overlays (player hitbox)
- Hearing radius (enemies)

UI Elements

- Objective text
- Win/Loss condition overlay
- Version label

10. Future Improvements

Future improvements on Boxu would include:

- Additional enemy archetypes
- External level loading
- Improved enemy path finding
- Expanded mission variety
- More advanced stealth feedback system
- Moveable objects

- Throwable objects (noise distraction used against enemy)